

2025

Penetration Testing

Gala (Wallet)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Gala team partnered with Hashlock to conduct a security Penetration Test of their Gala Project code base. Hashlock manually and proactively reviewed the code to ensure that the project's team and community the deployed tools were secure.

Project Context

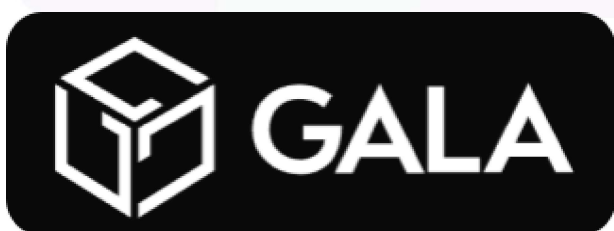
Gala is building a decentralized entertainment ecosystem that spans gaming, music, film, and blockchain technology. Their platform, GalaChain, is an L1 blockchain optimized for entertainment, offering fast transaction speeds, scalability, and energy efficiency. Gala provides developers with tools like Gala Creators and GalaChain Connect to create and deploy Web3 applications seamlessly. The ecosystem supports various projects, including games like Champions Arena and Spider Tanks, Gala Music for blockchain-based music distribution, and Gala Film for decentralized cinema experiences. Gala's mission is to empower creators and players by providing them with real ownership and control over their digital assets.

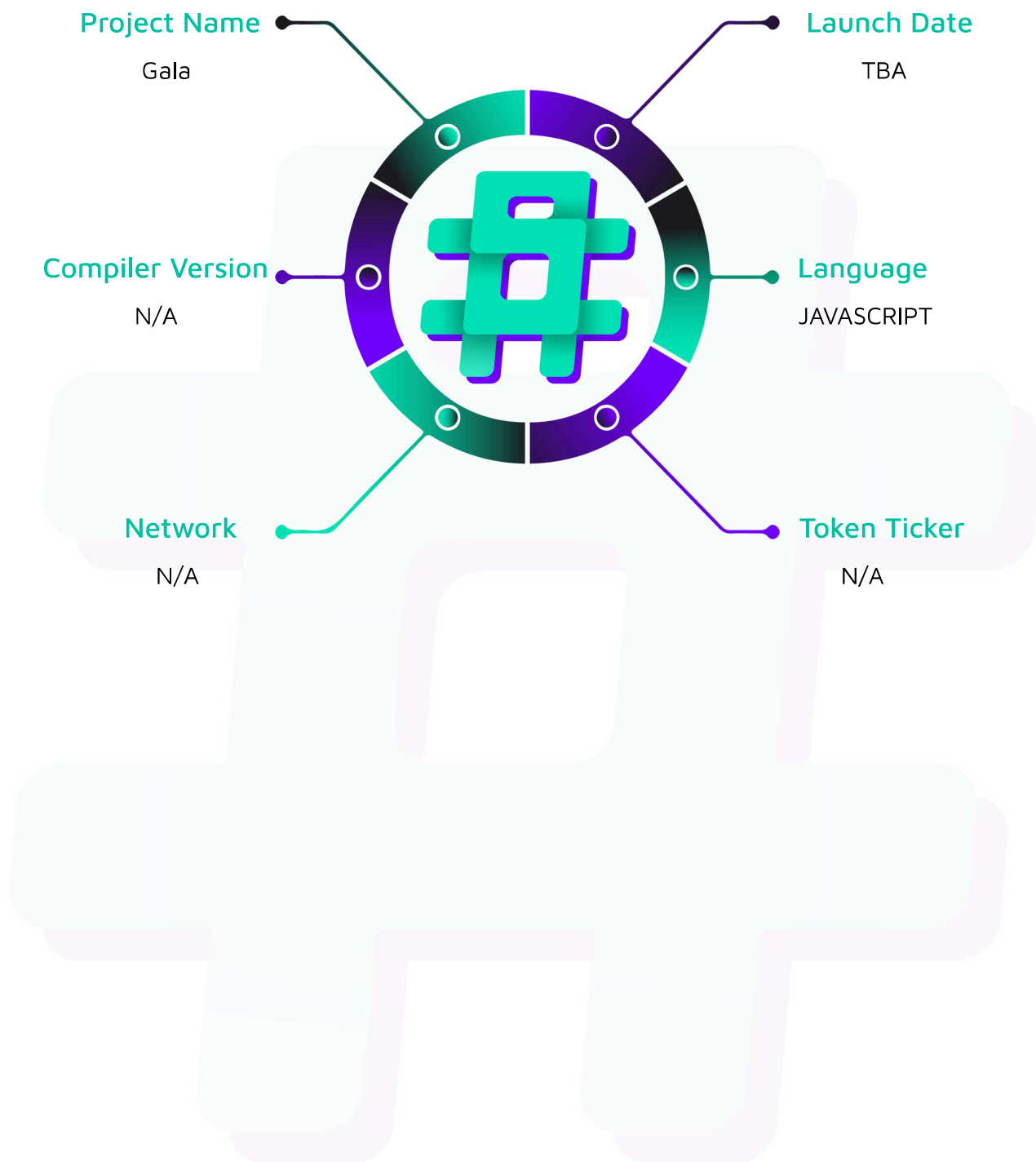
Project Name: Gala

Project Type: Wallet

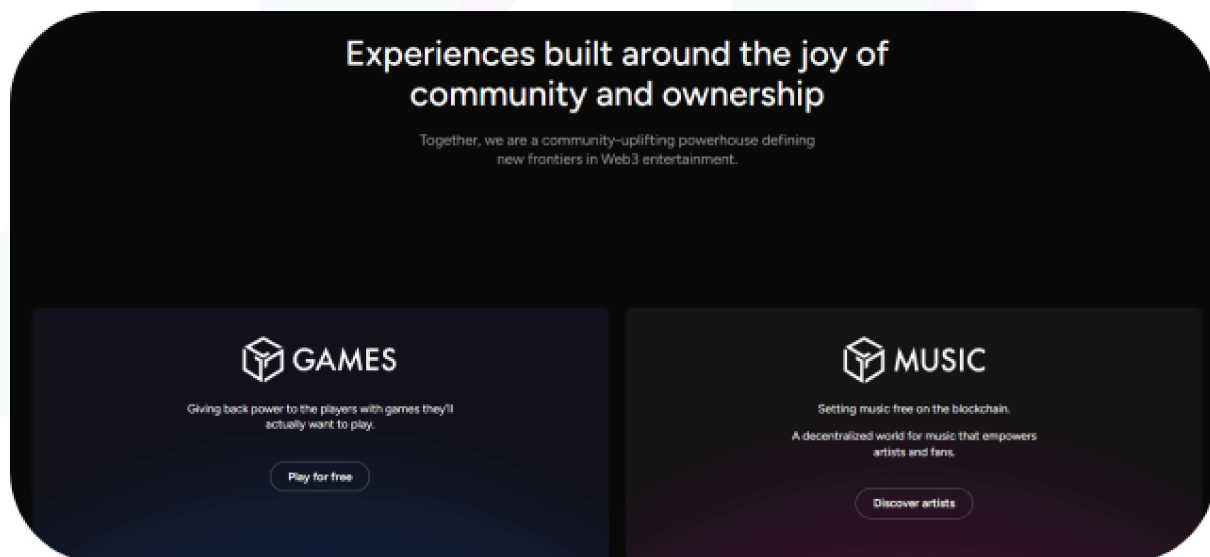
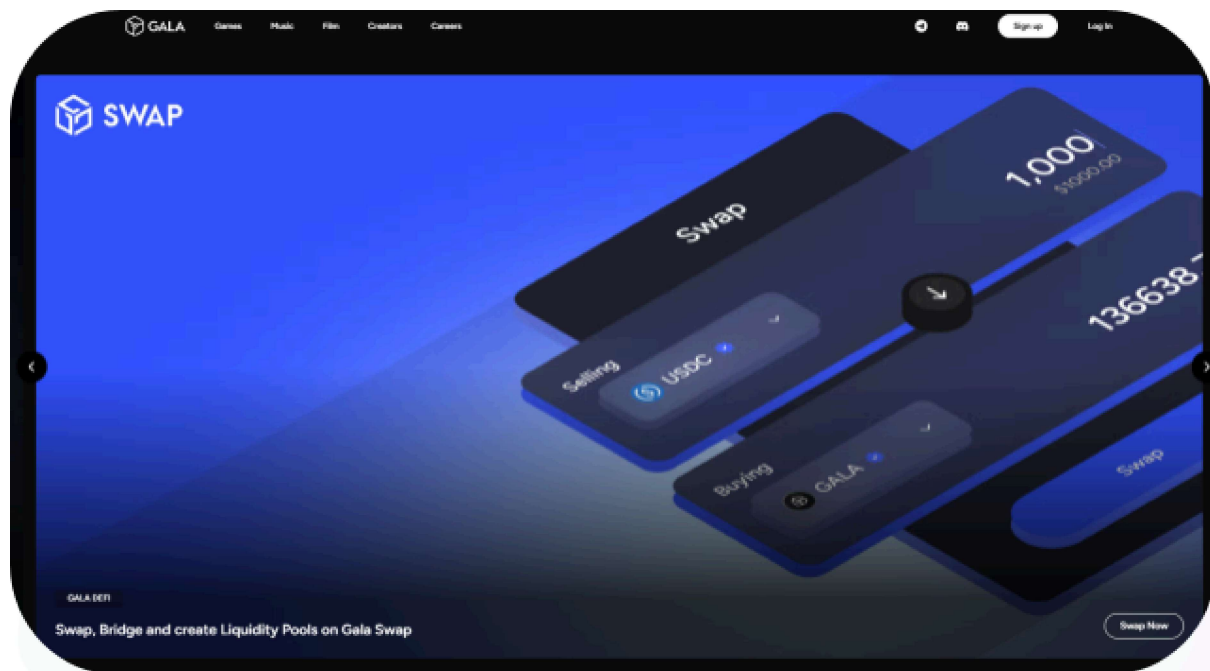
Website: gala.com

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Gala project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description			Gala Code Base
Platform			Web apps & APIs
Penetration Test Date			July 2025
Repository/Website URL/IP Tested			https://gitlab.com/gala-games/defi/wallet/wallet-extension/-/tree/bug/client_bug_20May
GitHub Commit Hash			5583149dea9a1b3d1ddafd4f353b67506c959d9f
Fix Commit Hash	Review	GitHub	63b4a6edc5e7f305b59f1bbdd6302283d5d6bca9

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High-severity vulnerabilities

4 Medium-severity vulnerabilities

5 Low-severity vulnerabilities

3 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] `src/helpers/wallet.helper.js#generateAndSetWallet` - Sensitive information logged to the console

Description

Across the codebase, there are multiple instances where sensitive information, such as mnemonics, user passwords, and private keys, is logged into the developer console with `console.log`:

- `src/helpers/wallet.helper.js:358`
- `src/helpers/wallet.helper.js:560`
- `src/helpers/wallet.helper.js:1297`
- `src/Pages/ExternalConnections/ExternalConnections.jsx:455`
- `src/Pages/ExternalConnections/ExternalConnections.jsx:480`
- `src/Pages/ManageWallet/ManageWallet.jsx:47`
- `src/Pages/ManageWallet/ManageWallet.jsx:544`
- `src/Pages/SecureWallet/SecureWallet.jsx:12`
- `src/Pages/ShowPrivateKeyChainSettings/ShowPrivateKey.jsx:23`
- `src/Pages/WelcomeScreens/ConfirmSecretPhrase.jsx:96`
- `src/Pages/WelcomeScreens/ConfirmSecretPhrase.jsx:97`
- `src/Pages/WelcomeScreens/SecretPhrase.jsx:17`
- `src/Pages/WelcomeScreens/SetPasswordScreen.jsx:122`
- `src/Pages/WelcomeScreens/SetPasswordScreen.jsx:147`

Impact

Anyone with access to the browser console, including users, malicious browser extensions, or third-party monitoring tools, can view sensitive information such as mnemonics or private keys. This exposure can lead to full wallet compromise and theft of user funds, as these secrets grant irreversible access to the user's assets.

Recommendation

Consider removing the logging of the above instances.

Status

Resolved

[H-02] public/inpage.js - Unvalidated Cross-Origin Message handling allows account manipulation

Description

The handler processes `switchAccount` requests from any origin without checking `event.origin`.

```
window.addEventListener('message', (message) => {  
  
  if (message.data) {  
  
    consoleLogFunc("inpage data::", message.data);  
  
  
    try {  
  
      if (message?.data?.data?.method === 'switchAccount') {  
  
        consoleLogFunc('User swithed his account');  
  
        window.gala.setAddress(message.data.data.address);  
  
        const event = new CustomEvent('accountChanged', {  
  
          detail: { address: message.data.data.address }  
  
        });  
  
      }  
  
    }  
  
  }  
  
});
```

Impact

A malicious page can call `window.postMessage({data:{data:{method:'switchAccount',address:'unwanted_account'}}}, '*')`

Recommendation

In the message listener, enforce `if (event.origin !== window.location.origin) return;` before processing.

Status

Resolved

[H-03] Swap.jsx, SwapDetails.jsx - No warning against extreme slippage may lead to the user losing funds

Description

The `handleSlippageChange` function in `Swap.jsx` allows users to set slippage tolerance up to 50% without any warnings or additional confirmations, exposing users to severe financial losses through sandwich attacks, shallow liquidity or volatility.

This is especially dangerous for less tech-savvy users who may not fully understand the internals of how slippage works and, as a result, be vulnerable to this kind of attack.

Vulnerability details

The `Swap.jsx::handleSlippageChange` function at line 126 accepts and validates user input for slippage tolerance:

```
const handleSlippageChange = (value) => {  
  
  const newValue = String(value).replace("%", "");  
  
  const regex = /^\\d*\\.?\\d{0,2}$/;  
  
  if (!regex.test(newValue)) {  
  
    return;  
  
  }  
  
  if (Number(newValue) >= 50) {  
  
    setSelectedSlippage("50");  
  
  }  
}
```

```

    setSlippageParam("50"); // Sets 50% as maximum with no warning

  } else if (newValue === "") {

    setSelectedSlippage("");

    setSlippageParam("0.5");

  } else if (Number(newValue) === 0) {

    setSelectedSlippage("0");

    setSlippageParam("0.5");

  } else {

    setSelectedSlippage(newValue);

    setSlippageParam(newValue);

  }

};

```

This slippage parameter flows directly into the swap execution in SwapDetails.jsx in the function amountOutMinimum, where it calculates the minimum output tokens:

```

const amountOutMinimum = useMemo(() => {

  return exponential_to_decimal(

    Number(wantedAmount) -

    (Number(wantedAmount) * Number(slippageParam)) / 100

  );

}, [wantedAmount, slippageParam]);

```

The final transaction at line 139 function sendResponse is executed with slippage up to 50%:

```

const sendResponse = await WalletHelper.sendSwap(

  bestQuote,

  account?.address,

```

```
amountOutMinimum, // Can be 50% less than expected  
  
amountInMaximum   // Can be 50% more than input  
  
);
```

With 50% slippage, a user expecting 1000 tokens would accept as few as 500 tokens, creating a massive profit opportunity for MEV bots to sandwich the transaction by manipulating the pool price between the user's transaction submission and execution. This may also occur with organic high volatility or when swapping within a low-liquidity pool.

While in some edge cases, or for professional users, this might be helpful, the vast majority of users should be clearly warned about slippage risk.

Impact

Unaware or less careful users can lose up to 50% of their swap value in a single transaction.

Recommendation

Implement a hard cap of 5% maximum slippage if users do not e.g. mark a checkbox that they know what they are doing. Add visual indicators showing the worst-case token amounts when slippage is set above standard thresholds.

Status

Resolved

[H-04] `src/helpers/wallet.helper.js` - Inconsistent encryption and decryption of account data

Description

The wallet helper is responsible for storing and retrieving user account data from `chrome.storage.local` and `chrome.storage.session`. The intended behavior is:

- `chrome.storage.local`: Persistent storage, must store encrypted account data using the user's password (e.g., `SecurityHelper.encrypt(finalAccounts, password)`). When retrieving accounts from the local storage, accounts must be decrypted before use (e.g., `SecurityHelper.decrypt(accounts, password)`).
- `chrome.storage.session`: Temporary runtime storage, should store plaintext account data for UI and internal logic.

However, several functions do not adhere to this convention:

- `unlockWallet`, `deleteWallet`, `setWalletPreference`, and `updateWalletName`: Retrieves encrypted accounts from local storage but does not decrypt them before use. Additionally, it incorrectly stores encrypted accounts in the session storage.
- `generateAndSetWallet`, `generateAndSetClientWallet`, and `generateAndSetWalletToRefreshToken`: store unencrypted accounts in the local storage, violating the encryption requirement.

For comparison, the `updateShowTotalBalance`, `updateBalanceInShow`, `updateTokens`, `generateAndSetWalletPrivateKey`, and `reorderWallet` functions implemented this correctly.

Impact

- If `chrome.storage.session` holds encrypted accounts, features relying on it (e.g., UI rendering, session restore) may break or behave unexpectedly.
- Unencrypted sensitive account data stored in `chrome.storage.local` may be accessible to other extensions or attackers with local access.

Recommendation

Consider implementing the following suggestions:

1. Encrypt accounts before storing in `chrome.storage.local`.
2. Decrypt accounts after retrieving from `chrome.storage.local`.
3. Do not encrypt accounts when writing to `chrome.storage.session`, as it should hold plaintext to support runtime operations.
4. Optionally implement utility wrappers (e.g., `getDecryptedAccounts()` and `setEncryptedAccounts()`) and utilize them throughout the codebase to minimize implementation errors.

Status

Resolved

Medium

[M-01] `src/Components/BalanceDetails/BalanceDetails.jsx#BalanceDetails` - Connected Dapps can be forcefully removed

Description

In lines 149-160 of `src/Components/BalanceDetails/BalanceDetails.jsx`, if the received message type is "GALA_DISCONNECT", the `setConnections([])` operation will be called to set the connected DApps to an empty array. This is problematic because if any Dapps want to disconnect, only the specific Dapp should be removed from the list of connected Dapps.

Impact

A DApp that disconnects itself will remove all connected DApps from storage. This can be triggered by any sender, including unauthorized ones, effectively allowing permissionless removal of all connections.

Recommendation

Consider updating the implementation to remove only the sender from the list of connected DApps, rather than clearing the entire list.

Status

Resolved

[M-02] src/helpers/wallet.helper.js - Hardcoded Authorization Bearer token**Description**

In line 2027 of `src/helpers/wallet.helper.js`, a hardcoded Authorization Bearer token is used directly in the headers to authenticate a request for logging events.

Impact

If an attacker gains access to the source code, they can reuse the token to gain unauthorized or privileged access to protected APIs or internal systems.

Recommendation

Consider storing the token as an environmental variable.

Status

Resolved

[M-03] public/manifest.json - Missing frame-ancestors allows clickjacking attacks**Description**

The current `content_security_policy` for `extension_pages` does not include any restrictions on how or where the extension's pages can be embedded.

This makes the extension vulnerable to clickjacking attacks, as its pages may be embedded in an external site via an `<iframe>`.

Impact

A malicious website could exploit this by overlaying invisible iframes and tricking users into interacting with the extension without their knowledge. This may lead to unintended actions such as approving a transaction, thereby stealing user funds.

Recommendation

Consider adding `"frame-ancestors 'none'"` to the `extension_pages` policy.

Status

Resolved

[M-04] src/helpers/wallet.helper.js - Mnemonics stored in localStorage**Description**

In `wallet.helper.js`, the `mnemonics` is expected to reside in `localStorage`. Despite being decrypted, this is an easy and extremely impactful target in case an XSS issue occurs.

```
static checkIfAccountExists = async (newMnemonics, password) => {  
  
  try {  
  
    // const { password } = await chrome.storage.session.get('password');  
  
    const { mnemonics } = await chrome.storage.local.get('mnemonics');
```

Impact

Any XSS or malicious extension can read and decrypt all wallet data.

Recommendation

Do not persist mnemonics in memory at all, especially in `localStorage`.

Status

Resolved

Low

[L-01] src/server.js - Redundant use of baseURL across the codebase

Description

In line 14, the Axios instance is initialized with `baseURL` set to `process.env.REACT_APP_BASE_URL`. This allows all HTTP requests that use the server variable to automatically include the base URL.

However, throughout the codebase, this base URL is still being manually defined as `${process.env.REACT_APP_BASE_URL}` when making requests, which are shown below:

- `src/CustomHooks/checkMaintenance.js:4`
- `src/helpers/wallet.helper.js:202`
- `src/helpers/wallet.helper.js:503`
- `src/helpers/wallet.helper.js:703`
- `src/helpers/wallet.helper.js:1438`
- `src/helpers/wallet.helper.js:1529`
- `src/helpers/wallet.helper.js:1900`
- `src/helpers/wallet.helper.js:2084`

Impact

If there are changes made to the `baseURL` in the `server.js` file, these hardcoded instances won't be updated automatically, potentially causing inconsistent behavior and bugs due to requests being sent to incorrect domains.

Recommendation

Consider removing all hardcoded `${process.env.REACT_APP_BASE_URL}` prefixes in the above instances and use relative paths when calling the server instance. This ensures centralized control of the base URL, reducing maintenance overhead.

Status

Resolved

[L-02] package.json - Vulnerable dependencies

Description

Running `npm audit` or `yarn audit` on the project reveals that several dependencies have known vulnerabilities. These vulnerabilities may exist in both direct and transitive dependencies used by the wallet application.

Impact

Outdated or vulnerable dependencies expose the application to publicly known exploits. Attackers can easily identify known vulnerabilities using publicly available databases and exploit them to compromise the application.

Recommendation

Consider updating all dependencies to their latest version and locking them.

Status

Resolved

[L-03] public/manifest.json - Broad host_permissions configuration**Description**

The extension currently requests `"https://*/*"` and `"<all_urls>"` in the `host_permissions`. This is problematic because the requested permissions are excessive, violating the ["Request minimal permissions"](#) best practice.

Impact

If the extension is compromised (e.g., through supply chain attacks, cross-site scripting (XSS), or vulnerable dependencies), the potential attack surface for attackers is significant.

For example, attackers could abuse these permissions to:

- Intercept user activity on any website
- Modify or exfiltrate content from any domain
- Inject malicious scripts or payloads

Additionally, these permissions bypass origin-level restrictions, granting the extension implicit trust over arbitrary domains and violating the principle of least privilege.

Recommendation

Consider minimizing the permissions by only registering APIs and websites that the wallet extension depends on. This would limit the attack surface for attackers if the extension were ever compromised.

Status

Resolved

[L-04] public/inpage.js - Lack of integrity protection when loading trust.js**Description**

The `TRUST_SCRIPT_URL` is used to load `trust.js` into the extension. While the origin is restricted to `source !== window`, in case of a compromise of `trust.js` hosting provider, it will impact all wallet users. There is no integrity verification of that script.

Impact

If the extension is compromised (e.g., through supply chain attacks, cross-site scripting (XSS), or vulnerable dependencies), the potential attack surface for attackers is significant.

Recommendation

Consider storing an integrity similar to how an [SRI attribute](#) is used to allow only certain versions of a script to be loaded.

Status

Resolved

[L-05] src/Components/CoinsTable/CoinsTable.jsx - Usage of dangerouslySetInnerHTML

Description

In src/Components/CoinsTable/CoinsTable.jsx, there is a code that displays price information:

```
<div  
  
    dangerouslySetInnerHTML={{ __html: ele.coinPrice }}  
  
></div>{" "}
```

dangerouslySetInnerHTML allows for rendering unvalidated HTML, potentially leading to XSS attacks.

Impact

Risk of XSS attacks if the coinPrice data input is compromised.

Recommendation

Use a secure display method, or cast to a number, whatever is in the price instead of displaying it directly.

Status

Resolved

QA

[Q-01] `src/helpers/wallet.helper.js` - Incorrect subevent name

Description

In line 1467 of `src/helpers/wallet.helper.js`, the `subeventName` for the `sendNft` operation is hardcoded as `sendTokens`. This is incorrect because it should be `sendNft`.

Similarly, the subevent name for `src/Pages/Nft/PreviewNft.jsx:155` should be replaced with `completeNft`.

Recommendation

Consider updating the subevent names correctly.

Status

Acknowledged

[Q-02] `src/Pages/AddresBook/AddresBook.jsx` - Spelling errors

Description

`src/Pages/AddresBook/AddresBook.jsx` contains a spelling error in the directory and filename.

Recommendation

Consider updating it to be `src/Pages/AddressBook/AddressBook.jsx`.

Status

Resolved

[Q-03] inpage.js - Infura API key exposure**Description**

There is a hardcoded Infura API key in `inpage.js`. While this may be intended, it should be verified whether it is so.

Recommendation

Verify if the key disclosure is intended.

Status

Resolved

Conclusion

After Hashlock's analysis, the Gala project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.